Pregled dokumentacije

Ugotovila sem, da README ne omogoča zanesljive ponovitve namestitve v novem okolju:

- Glavna neskladja in napake:
    - Spremenljivka EXPO_PUBLIC_WEB_APP_URL nima učinka, saj ima App.js URL trdo kodiran na produkcijo, zato lokalni razvoj spletnega dela v Expu ne deluje.
    - Neusklajene verzije Node.js (README navaja 18+, funkcije 24, CI teče na 20).
    - Neenotnost orodij: konflikt med npm in pnpm (dve lock datoteki, zastarela navodila v CLAUDE.md).
    - Nekateri ukazi za namestitev v README so odvečni, saj so paketi že v package.json.
    - Nepotrebna konfiguracija, to je neuporabljen Gemini API ključ v CI.

- Manjkajoče začetno stanje:
    - Vse je vezano na obstoječi projekt mindmend-a8839. Ni navodil za nastavitev lastnega Firebase projekta ali Firestore varnostnih pravil (firestore.rules).
    - Testni računi in podatki obstajajo le v produkcijski bazi, ni pa skripte za začetne podatke.

- Operacijski sistemi:
    - Ukaz cp v README ne deluje v Windows cmd (potreben je copy ali bash).
    - Razlike pri globalnih namestitvah (sudo na Linux/macOS vs. Execution Policy v PowerShellu).
    - Windows požarni zid in WSL pogosto blokirata povezavo telefona/Expo z lokalnim strežnikom.
    - Android emulator zahteva naslov 10.0.2.2, iOS simulator deluje le na macOS.

Standardi kodiranja

Projekt nima formalno zapisanih standardov. Opažena odstopanja v kodi:

- Enkrat enojni, drugič dvojni narekovaji.
- Izklopljen strog način za TypeScript (strict: false) ter veliko primerov uporabe tipa any.
- Mešanje slovenščine in angleščine v kodi, imenih funkcij in komentarjih.
- Nekonsistentno poimenovanje datotek, včasih PascalCase včasih camelCase.
- V projektu je ostalo tudi veliko zakomentirrane kode, ki ni bila izbrisana.

- Predlagani standardi ekipe:
    - Angleščina za vso kodo, komentarje in tipe, saj zaenkrat prevladuje v projektu.
    - Poimenovanje komponent v PascalCase, funkcije v camelCase, tipi v PascalCase..

- Definiranje globalnih konstant v UPPER_SNAKE.
- Brez any
- Odstranitev console.log iz produkcije.

Testiranje

Pri enotnih testih storitev je izvedenih 20 testov, ki pokrivajo 5 od 11-ih datotek v mapi services. Pokrita so področja terminov, uporabnikov, vsebin, kartotek strank in plačil, medtem ko so avtentikacija, onboarding in Gemini nepokriti. Enotni testi pomožne logike so prav tako izvedeni s 17 testi, ki preverjajo vsa 4 orodja v mapi utils. To so termini, serije in interakcije. Testi komponent in zaslonov niso bili izvedeni, čeprav bi morali biti, da bi preverili pravilno delovanje uporabniškega vmesnika. Prav tako niso bili izvedeni testi zalednih funkcij (Cloud Functions), ki bi bili nujno potrebni za zanesljivo in varno delovanje logike plačil, vračanja denarja in pošiljanja notifikacij. CI/CD cevovod je vpeljan le delno, saj pokriva zgolj spletni del projekta (web-template), medtem ko mobilna aplikacija v Expo in zaledne funkcije vanj niso vključene. Integracijski, end-to-end in varnostni testi niso bili izvedeni, morali pa bi biti za preverjanje celostnega delovanja sistema med frontendom in backendom, varnostnih pravil ter pravilne avtorizacije med vlogami uporabnika in terapevta.
Prav tako je v CI cevovodu je nastavljen continue-on-error: true, kar pomeni, da neuspešni testi in napake ne ustavijo objave na produkcijo. Logika plačila in izplačila pa sploh nima avtomatskih testov.

Pri izdelavi naloge sem si pomagala z orodjem generativne umetne inteligence pri analizi posebnosti namestitve na različnih operacijskih sistemih ter za odkritje neskladij med uporabljenimi različicami odvisnosti in orodij.